

Simulador de blockchain: Proof of Work y Proof of Stake en una aplicación web

Guía de entrega del proyecto

Facultad de Ingeniería · Dr. José de Jesús Ángel Ángel

Fecha de entrega: [por definir] · Equipos: [por definir]

1 Objetivo

Construir una aplicación web en **Python + Flask** que simule una blockchain con entre **10 y 20 nodos** y que funcione en dos versiones:

V1. Proof of Work (PoW): los nodos son mineros que compiten por encontrar un hash válido.

V2. Proof of Stake (PoS): los nodos son validadores que apuestan, se sortea un proponente y el bloque se somete a votación.

Ambas versiones comparten la misma estructura de bloque, las mismas transacciones firmadas y la misma regla de difusión: cada nodo conserva su propia copia de la cadena y *valida* lo que recibe antes de aceptarlo.

Criterio central de evaluación. La aplicación debe soportar cualquier uso razonable o erróneo sin caerse ni quedar en un estado inconsistente. Un error no controlado (traceback, error 500 o cadena corrupta) durante la revisión es una falla costosa (sección 8).

2 Requisitos comunes a ambas versiones

2.1 Nodos y cuentas

- El número de nodos N se elige en la interfaz y debe cumplir $10 \leq N \leq 20$; cualquier otro valor se rechaza con un mensaje claro.
- Cada nodo tiene un identificador, un par de claves (privada/pública), un saldo y *su propia copia* de la cadena.
- No se pide red P2P real: la “red” puede simularse en memoria, si se hace la simulación en una conexión local es un valor agregado.

2.2 Transacciones firmadas

Una transacción contiene emisor, receptor, monto y marca de tiempo. El emisor la firma con su clave privada; cualquier nodo la verifica con la clave pública. Se recomienda `ecdsa` (curva SECP256k1) o `cryptography` (Ed25519). Se firma la serialización canónica de la transacción:

```
def serializar(tx):          # orden fijo de llaves = misma firma en todos lados
    datos = {k: tx[k] for k in ("emisor", "receptor", "monto", "timestamp")}
    return json.dumps(datos, sort_keys=True, separators=(",", ":")).encode()

firma = clave_privada.sign(serializar(tx)).hex()
```

2.3 Estructura del bloque

Campos mínimos del bloque

Campo	Descripción
numero	Altura del bloque; el génesis es el 0.
timestamp	Momento de creación.
transacciones	Al menos una transacción hecha por un usuario.
firma	Firma digital del emisor sobre cada transacción (con su clave pública).
hash_anterior	Hash del bloque previo; encadena el libro.
nonce	En PoW, el número que hace válido el hash; en PoS puede fijarse en 0.
hash	SHA-256 de todos los campos anteriores, en forma canónica.
proponente	Minero ganador (PoW) o validador sorteado (PoS).
votos	<i>Sólo PoS</i> : quién votó y con qué peso.

```
def hash_bloque(b):
    # el campo "hash" nunca entra en su propio calculo
    datos = {k: v for k, v in b.items() if k != "hash"}
    return hashlib.sha256(json.dumps(datos, sort_keys=True).encode()).hexdigest()
```

2.4 Validación y difusión

Un nodo acepta una cadena recibida sólo si *todas* estas condiciones se cumplen; en caso contrario la rechaza y conserva la suya:

- cada **hash** coincide con el recalculado y cada **hash_anterior** con el del bloque previo;
- todas las firmas de transacciones son válidas y ningún emisor gasta más de lo que tiene (incluido el doble gasto dentro del mismo bloque);
- se cumple la regla del consenso (hash bajo el objetivo en PoW; votos suficientes en PoS);
- la cadena recibida es más larga que la propia (regla de la cadena válida más larga).

La interfaz debe mostrar, para cada nodo, la altura de su cadena y el hash de su último bloque, de modo que se vea cuándo todos están sincronizados.

3 Versión 1: Proof of Work

Lo que debe ocurrir en cada bloque

- Se toma al menos una transacción firmada de la lista de pendientes.
- Los N nodos se vuelven mineros y buscan un nonce tal que el hash del bloque empiece con d ceros hexadecimales (d configurable; sugerido $3 \leq d \leq 5$).
- Nonces disjuntos:** el minero i (de 0 a $N - 1$) prueba únicamente i , $i + N$, $i + 2N$, ...; dos mineros nunca prueban el mismo nonce.
- La competencia se ve:** la interfaz muestra en vivo, por minero, el nonce actual, el número de intentos y el último hash calculado.
- Al haber ganador, todos se detienen.** El ganador agrega el bloque a su cadena y la envía a los demás nodos, que la validan.
- Recompensa con 6 confirmaciones:** la recompensa del bloque de altura h queda *pendiente* y se acredita al saldo del ganador cuando la cadena alcanza la altura $h + 6$. La interfaz distingue saldo disponible y recompensas pendientes.

Sugerencia de proceso

La forma más sencilla de *ver* la competencia es simular por **rondas** (*ticks*) en lugar de usar hilos: en cada ronda cada minero prueba k nonces y el navegador consulta el estado con **fetch** cada 200–500 ms.

```
def ronda_pow(bloque, mineros, N, k, d):
    for m in mineros:                                # m.i en 0..N-1
        for _ in range(k):
            bloque["nonce"] = m.nonce
            h = hash_bloque(bloque)
            m.intentos += 1; m.ultimo_hash = h
            if h.startswith("0" * d):
                return m, h                            # ganador: se detiene la ronda
            m.nonce += N                                # i, i+N, i+2N, ...
    return None, None
```

Si prefieren hilos, usen un `threading.Event` para detener a todos y un `Lock` para que sólo un ganador pueda registrar el bloque. En ambos casos deben definir y documentar qué pasa si dos mineros encuentran un nonce válido en la misma ronda (por ejemplo, gana el hash menor).

La simulación debe poder minar al menos **10 bloques seguidos**, para que se observe cómo maduran las primeras recompensas.

4 Versión 2: Proof of Stake

Lo que debe ocurrir en cada bloque

1. **Elección de validadores:** para cada bloque nuevo se elige un conjunto de validadores entre los nodos (todos los que tengan saldo, o un subconjunto aleatorio de tamaño configurable).
2. **Apuestas:** cada validador apuesta una cantidad $0 < a_i \leq$ su saldo, que queda bloqueada durante la ronda.
3. **Sorteo del ganador:** se elige un proponente con probabilidad proporcional a su apuesta, $P(i) = a_i/A$ con $A = \sum a_j$, usando un número aleatorio reproducible.
4. **Bloque candidato:** el ganador construye el bloque con las transacciones pendientes y lo firma.
5. **Votación:** cada validador revisa el bloque y vota; su voto pesa según su valor acumulado (apuesta). El bloque se acepta si los votos a favor suman al menos $\frac{2}{3}$ de A , es decir, si $3V \geq 2A$.
6. **Si se acepta:** se agrega a la cadena, se difunde a todos los nodos, se paga la recompensa y se liberan las apuestas.
7. **Si se rechaza:** el proponente pierde su apuesta completa o un porcentaje según el valor de la transacción (ver abajo), y se repite el sorteo con los validadores restantes.

Castigo por rechazo. Elijan una de estas reglas, háganla configurable y justifíquela en el reporte:

$$(A) \quad c = a_p \qquad (B) \quad c = \min(a_p, \alpha \cdot \text{valor de las transacciones}), \quad 0 < \alpha \leq 1.$$

Para que el rechazo ocurra en la demostración, la aplicación debe permitir marcar un nodo como *deshonesto*: cuando lo sortean, propone un bloque con una firma alterada o un gasto mayor al saldo, y los validadores honestos votan en contra.

Sugerencia de proceso

Sorteo ponderado con una semilla pública y reproducible (es el mismo esquema de intervalos acumulados del ejemplo mínimo visto en clase):

```
def sortear(validadores, hash_anterior, numero, intento):
    semilla = f"{hash_anterior}|{numero}|{intento}".encode()
    A = sum(v.apuesta for v in validadores)
    r = int(hashlib.sha256(semilla).hexdigest(), 16) % A
    acum = 0
    for v in validadores:
        acum += v.apuesta
        if r < acum:
            return v
```

orden fijo, p. ej. por id

Conviene modelar la ronda como una pequeña máquina de estados: APUESTAS → SORTEO → CANDIDATO → VOTACION → ACEPTADO / RECHAZADO, y mostrar el estado actual en la interfaz.

5 Casos que la aplicación debe contemplar

La revisión incluirá probar en vivo los casos siguientes. Cada uno debe producir un mensaje claro en la interfaz y dejar el sistema consistente.

Lista mínima de casos

Ámbito	Caso
Entradas	N fuera de 10–20; montos negativos, cero, no numéricos o vacíos; emisor igual a receptor; nodos inexistentes; dificultad fuera de rango.
Transacciones	Firma alterada; transacción firmada por otra clave; saldo insuficiente; doble gasto en el mismo bloque o en bloques distintos; lista de pendientes vacía al intentar minar o proponer.
Cadena	Bloque con hash o <code>hash_anterior</code> alterado; cadena recibida más corta o inválida; manipular un bloque intermedio y verificar que la cadena completa se rechaza.
PoW	Dos ganadores en la misma ronda; pedir minar mientras ya se mina; dificultad que no se resuelve en tiempo razonable (límite de rondas o cancelación); recompensa consultada antes de 6 confirmaciones; recompensa falsa (monto distinto al establecido).
PoS	Ningún validador con saldo; apuesta mayor al saldo, cero o negativa; votación exactamente en $\frac{2}{3}$; voto de un nodo que no es validador o que vota dos veces; proponente deshonesto; todos los validadores castigados hasta quedar sin saldo.
Aplicación	Reiniciar la simulación; recargar la página a mitad de una ronda; dos pestañas enviando acciones a la vez; peticiones directas a las rutas con datos mal formados.

6 Sugerencias de organización

Estructura del proyecto

```
simulador/  
  app.py           # rutas Flask; solo recibe, valida entradas y responde JSON  
  nucleo/  
    cripto.py      # claves, firmar, verificar  
    bloque.py      # Transaccion, Bloque, hash_bloque, validar_cadena  
    red.py         # Nodo, difundir(cadena) -> cada nodo valida  
    pow.py         # ronda_pow, recompensas pendientes  
    pos.py         # apuestas, sortear, votar, castigar  
  templates/ static/ # interfaz y JavaScript (fetch periodico)  
  tests/          # pytest: un test por caso de la seccion 5  
  README.md requirements.txt
```

- **Separen la lógica de la web.** Todo el consenso debe poder ejecutarse y probarse sin Flask; las rutas sólo validan entradas y llaman al núcleo.
- **Validen en el servidor**, aunque el formulario ya valide en el navegador: la revisión enviará peticiones directas a las rutas.
- **Usen una semilla configurable** para el azar. Con la misma semilla, la simulación debe repetirse igual; eso facilita depurar y evaluar.
- **Registren eventos** (bloque minado, cadena rechazada, castigo, recompensa acreditada) en una bitácora visible en la interfaz.
- **Escriban primero las pruebas** de los casos de la sección 5; después la interfaz.

7 Entregables

1. Repositorio con el código, `requirements.txt` y un `README` con instrucciones para correr la aplicación en una máquina limpia (`pip install -r requirements.txt` y `flask run`).
2. Pruebas automatizadas (`pytest`) que cubran la lista de casos.
3. Reporte breve (máximo 4 páginas): diseño, regla de desempate en PoW, regla de castigo en PoS y casos contemplados.
4. Video de 5 a 8 minutos que muestre ambas versiones, un rechazo en PoS y la maduración de una recompensa en PoW, dar la liga del video en reporte.

8 Rúbrica

Criterio	Peso	Completo (100 %)	Parcial (50 %) / Insuficiente (0 %)
Robustez y casos contemplados	35	Todos los casos de la sección 5 responden con un mensaje claro; ninguna acción produce error 500 ni deja la cadena o los saldos inconsistentes; las pruebas los cubren.	Parcial: algunos casos sin mensaje o sin prueba. Insuficiente: la aplicación se cae o se corrompe con entradas comunes.
Bloque, firmas y validación	15	El bloque tiene todos los campos; hash y firmas se verifican; cada nodo valida la cadena completa antes de aceptarla.	Faltan campos, o los nodos aceptan cadenas sin validarlas.
Versión PoW	20	Entre 10 y 20 mineros con nonces disjuntos; competencia visible; todos se detienen al haber ganador; difusión correcta; recompensa tras 6 confirmaciones.	Falta la visualización, el paro o la maduración; nonces repetidos entre mineros.
Versión PoS	20	Validadores y apuestas por bloque; sorteo ponderado reproducible; votación por valor con umbral de $\frac{2}{3}$; castigo aplicado; nuevo sorteo tras rechazo.	El sorteo no es ponderado, la votación no pesa por valor o el rechazo no tiene consecuencias.
Interfaz web	5	Se ve el estado de cada nodo, la sincronización y la bitácora de eventos.	Interfaz mínima o confusa.
Reporte y video	5	Explican el diseño y las reglas elegidas, y muestran todos los escenarios pedidos.	Incompletos o sin justificar las decisiones.
Total	100		